Agents as Architecture Evaluators: Automating ATAM and SAAM for Continuous Assessment

Adyansh Kakran✉
SERC, IIITH
Hyderabad, India
adyansh.kakran@research.iiit.ac.in

Sathvika Miryala
SERC, IIITH
Hyderabad, India
miryala.sathvika@research.iiit.ac.in

Karthik Vaidhyanathan
SERC, IIITH
Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in

Abstract

The long-term success of a software system relies heavily on its underlying architecture, which dictates its performance with respect to quality attributes. Software Architecture Evaluation methods, such as the Architecture Tradeoff Analysis Method (ATAM) and the Software Architecture Analysis Method (SAAM), are widely recognized for their ability to ensure reliable quality attributes. Their adoption remains limited due to cost, time, and expertise. We investigate agentic AI to lower these barriers. From the literature, we built a dataset of 39 cases and an agentic workflow that synthesizes evaluation reports from high-level architectural descriptions. Automated metrics and human evaluation yield relevance and completeness of **4.5/5** and **3.9/5**. Agents automated scenario generation and risk identification, under **5 minutes** at less than **$0.50** per run.

CCS Concepts

• **Software and its engineering → Software maintenance tools**.

Keywords

Software Architecture Evaluation, LLMs, Agentic AI

ACM Reference Format:
Adyansh Kakran, Sathvika Miryala, and Karthik Vaidhyanathan. 2026. Agents as Architecture Evaluators: Automating ATAM and SAAM for Continuous Assessment. In *Proceedings of the 1st International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3843282.3844431

1 Introduction

Software architecture serves as the fundamental bridge between high-level requirements and technical implementation. Evaluating architecture is essential for ensuring quality attributes such as performance, security, and maintainability are addressed. Established evaluation methods like ATAM and SAAM have been used to identify architectural risks and manage the complex tradeoffs [3, 4].

Industrial adoption of these methods remains limited due to high resource demands [1], and synchronous multi-day stakeholder participation is often incompatible with agile processes, so organizations omit formal evaluation and silently accumulate architectural debt. Earlier automation used quantitative metrics such as coupling and cohesion, but rule-based approaches cannot capture the semantic nuances needed to reason about quality-attribute tradeoffs against business drivers. Large Language Models (LLMs) offer a path toward automation. Capilla et al. [2] analyzed Microsoft Copilot for conducting ATAM evaluations using single LLM interactions. To overcome the reasoning limits of single models, recent work has turned to multi-agent systems that assign distinct roles and simulate collaboration, yet existing agentic workflows in software engineering focus predominantly on code generation rather than high-level architectural assessment. To the best of our knowledge, no prior work has leveraged agentic workflows to automate the multi-stakeholder dynamics of ATAM and SAAM. We propose a multi-agent workflow to simulate a human evaluation team. The system takes structured architectural summaries as input and synthesizes evaluation reports. Our results indicate that while AI agents replicate evaluation and excel at scenario generation and broad risk identification, they struggle to filter business-irrelevant findings.

2 Study Design and Approach

Dataset Curation. To evaluate our approach, we required architectural descriptions and corresponding human conducted evaluations. To the best of our knowledge, there is currently no publicly available dataset that maps architectural descriptions to their corresponding evaluations. A Google Scholar search was conducted creating dataset of 39 case studies. We designed a JSON schema for both ATAM and SAAM to convert unstructured articles into a standardized format. The schema separates data into two sections: **Evaluation Inputs**, **Ground Truth Outputs**. We employed a context-aware few-shot prompting strategy. Two of the authors verified every generated JSON file against the source for factuality, completeness, and data leakage.

Agentic Evaluation Framework. Figure 1 illustrates the architecture of the agentic framework. The workflow is governed by *Orchestrator* that manages evaluation state and enforces task sequencing. Four specialized LLM agents communicate strictly through a shared JSON blackboard: the *Customer* provides architectural information and clarifies domain constraints; the *Manager* translates stakeholder inputs into formal requirements; the *Architect* performs technical analysis and formalizes system descriptions; and the *Evaluation Team* identifies risks, trade-offs, and final results.

Experimental Setup. Persona-based prompting enforces role boundaries: the Architect is prompted to be defensive and technical, while the Evaluation Team is skeptical and critical. Gemini 3 Flash, Claude 4.5 Sonnet, and DeepSeek v3.2 were used for the agentic workflow experimentation. [1]

[1] Detailed study design and code is available at https://zenodo.org/records/22144864.



This work is licensed under a Creative Commons Attribution 4.0 International License.
AgenticDev '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2985-0/2026/10
https://doi.org/10.1145/3843282.3844431

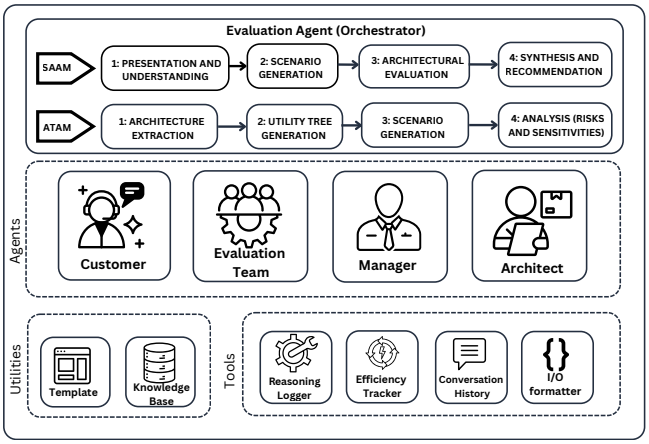


Figure 1: Agentic Architecture for both Workflows

3 Preliminary Results and Conclusions

Research Questions. The experiments are designed to address the following research questions.

RQ1: *How accurately can LLMs extract unstructured architectural data and populate formal evaluation templates?*

RQ2: *To what extent can agentic workflows replicate human-led software architecture evaluations?*

- **RQ2.1:** *How do agent-generated evaluations compare to human baselines across automated metrics?*
- **RQ2.2** *How does the quality of agentic evaluations differ from human teams in terms of quality, as judged by humans?*

RQ3: *What are the cost and sustainability implications of automating software architecture evaluation?*

Findings. **RQ1. Extraction produced zero JSON schema violations, but 18% of cases needed missing information added and 6% contained hallucinations.** Authors inserted niche scenarios that were present in the source but omitted from the template. In the hallucination cases, the model filled fields that the prompt instructed to mark as "Unknown" when the original was ambiguous.

RQ2.1. Agent evaluations were similar in sense to human baselines (BERTScore 0.55–0.60**), with Claude stronger on completeness and Gemini stronger on conciseness.** ROUGE-L, BLEU, and METEOR were low because they measure word overlap, so they mainly help detect leakage rather than evaluation quality. Proprietary models held a slight edge over DeepSeek v3.2. The Claude–Gemini split is a verbosity bias: more complete reports listed more risks, while Gemini's shorter outputs were denser.

RQ2.2. Agents replicate the form of an evaluation but differ in substance, listing more risks than humans and struggling to filter business-irrelevant findings. Claude 4.5 Sonnet produced the most complete evaluations on our dataset. Human reviewers penalized conciseness when low-impact items, such as dependency version mismatches, sat alongside structural concerns. Agents also surfaced risks absent from some source studies.

RQ3. The most expensive agentic run cost about $0.40 and finished in under 5 minutes, compared with 3 to 4 person-weeks and $5,000–$10,000 for a human ATAM. Gemini 3 Flash used the fewest tokens and the least time while receiving the highest human-evaluation ratings. Claude 4.5 Sonnet was the most verbose, taking nearly twice as long. The workflow is not a full replacement for human expertise, but the cost reduction lets evaluation be treated as a routine check rather than a one-time ritual.

Table 1: Scores for generated evaluations. Rel.: Relevance; Coh.: Coherence; Comp.: Completeness; Conc.: Conciseness.

		LLM-as-a-Judge				Human Evaluation			
Method	**Model**	**Rel.**	**Coh.**	**Comp.**	**Conc.**	**Rel.**	**Coh.**	**Comp.**	**Conc.**
SAAM	Gemini	3.74	4.43	3.18	**4.19**	**4.38**	4.29	3.71	**4.54**
	DeepSeek	3.89	4.65	3.24	4.10	4.29	**4.38**	3.75	4.29
	Claude	**4.47**	**4.83**	**4.23**	3.89	3.92	4.29	**3.88**	2.96
ATAM	Gemini	4.44	**4.93**	3.58	**4.88**	**4.55**	**4.23**	3.46	**4.36**
	DeepSeek	4.25	4.71	3.31	4.43	4.14	3.86	3.18	3.86
	Claude	**4.64**	4.72	**4.25**	4.14	4.18	4.14	**3.64**	3.18

Discussion and Conclusions. Agents perform well at broad risk identification but struggle with relevance filtering. Human evaluation noted that outputs often contained too many scenarios, and agents frequently prioritized minor details. Output quality was heavily dependent on the description of the input: in the SAAM workflow, agents often defaulted to non-functional requirements when the input context was sparse, whereas the ATAM workflow demonstrated significantly higher quality when the Evaluation Inputs included raw candidate scenarios. These findings support using agentic systems as drafting tools rather than independent evaluators. The dataset curated in this study provides a starting point for validating specialized models. Practitioners should adopt these systems not as replacements, but as assistants that perform the heavy lifting of scenario generation and initial risk scanning, with human architects remaining responsible for filtering the data and making the final decisions. By establishing a validated dataset and designing role-based agents, we demonstrated that LLMs in an agentic setup can effectively replicate the procedural steps of ATAM and SAAM. While agents significantly reduce the time and cost associated with evaluation, they currently function best as helpers rather than replacements, enabling a transition from sporadic, high-cost reviews to continuous architectural assessment. Future work includes expanding the dataset, supporting other evaluation methods, and turning the *Customer* into a human interlocutor.

References

[1] Muhammad Ali Babar, Len Bass, and Ian Gorton. 2007. Factors Influencing Industrial Practices of Software Architecture Evaluation: An Empirical Investigation. In *Software Architectures, Components, and Applications*. Springer Berlin Heidelberg.

[2] Rafael Capilla, J. Andrés Díaz-Pace, Yamid Ramírez, Jennifer Pérez, and Vanessa Rodríguez-Horcajo. 2025. Supporting architecture evaluation for ATAM scenarios with LLMs. arXiv:2506.00150 [cs.SE] https://arxiv.org/abs/2506.00150

[3] R. Kazman, L. Bass, G. Abowd, and M. Webb. 1994. SAAM: a method for analyzing the properties of software architectures. In *Proceedings of 16th International Conference on Software Engineering*. 81–90. doi:10.1109/ICSE.1994.296768

[4] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, and J. Carriere. 1998. The architecture tradeoff analysis method. In *Proceedings. Fourth IEEE International Conference on Engineering of Complex Computer Systems (Cat. No.98EX193)*. 68–78. doi:10.1109/ICECCS.1998.706657

Received 2026-07-15; accepted 2026-08-20